

1 Задания 1 и 2. Разработка адаптивного интерфейса и MVC-каркаса

В рамках первого этапа был разработан адаптивный Landing Page для образовательной платформы «EduSchool». Согласно требованиям технического задания, при верстке категорически не использовался тег `<table>` и абсолютные единицы измерения (px).

1.1 Реализация адаптивной сетки (CSS Grid)

Для отображения прайс-листа курсов был применен подход CSS Grid, обеспечивающий корректное отображение на мобильных устройствах. Для корректной работы скринридеров были добавлены семантические ARIA-роли (`role="table"`, `role="row"`).

```
171 .grid-table {
172     display: flex;
173     flex-direction: column;
174     background: var(--bg-white);
175     border: var(--border-thin);
176     border-radius: var(--radius);
177     overflow: hidden;
178 }
179
180 .table-header,
181 .table-row {
182     display: grid;
183     grid-template-columns: 2fr 1fr 1fr 1fr;
184     border-bottom: var(--border-thin);
185 }
```

Рис. 1: Скриншот 1: Реализация адаптивной таблицы через CSS Grid

1.2 Каркас Spring Boot MVC

Был инициализирован проект на базе фреймворка Spring Boot. Реализован паттерн MVC (Model-View-Controller). Маршрутизация настроена через `HomeController` и `CourseController`, а статический HTML интегрирован в шаблонизатор Thymeleaf.



Рис. 2: Скриншот 2: Структура каталогов MVC-приложения

1.3 Валидация форм (Регулярные выражения)

На стороне клиента (Vanilla JavaScript) реализована строгая валидация полей ввода с помощью регулярных выражений.

```

447 <script>
448   document.addEventListener('DOMContentLoaded', () => {
449     const form = document.getElementById('leadForm');
450
451     const regexPatterns = {
452       name: /^[А-Яа-яЁёА-Za-z\s-]{2,50}$/ ,
453       email: /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/ ,
454       phone: /^\+7 \(\d{3}\) \d{3}-\d{2}-\d{2}$/ ,
455       date: /^(0[1-9]|[12][0-9]|3[01])\.(0[1-9]|1[0-2])\.(20[0-2][0-9]|2030)$/
456     };
457
458     const errorMessages = {
459       name: 'Имя должно содержать от 2 до 50 букв.',
460       email: 'Введите корректный E-mail адрес.',
461       phone: 'Формат телефона: +7 (XXX) XXX-XX-XX',
462       date: 'Формат ДД.ММ.ГГГГ. Год должен быть от 2000 до 2030.'
463     };
464
465     const validateField = (fieldId) => {
466       const input = document.getElementById(fieldId);
467       const errorSpan = document.getElementById(`${fieldId}Error`);
468       const value = input.value.trim();
469       const isValid = regexPatterns[fieldId].test(value);
470
471       if (!isValid) {
472         input.classList.add('invalid');
473         errorSpan.textContent = errorMessages[fieldId];
474         errorSpan.style.display = 'block';
475         return false;
476       } else {
477         input.classList.remove('invalid');
478         errorSpan.style.display = 'none';
479         return true;
480       }
481     };
482
483     form.addEventListener('submit', (e) => {
484       e.preventDefault();
485
486       const isNameValid = validateField('name');
487       const isEmailValid = validateField('email');
488       const isPhoneValid = validateField('phone');
489       const isDateValid = validateField('date');
490
491       if (isNameValid && isEmailValid && isPhoneValid && isDateValid) {
492         alert('Форма отправлена, заявка принята.');
```

Рис. 3: Скриншот 3: JavaScript логика клиентской валидации

Детальный разбор разработанных регулярных выражений:

- Имя родителя: `^[А-Яа-яЁёА-Za-z\s-]{2,50}$`

- $\wedge$ и $\$$ — начало и конец строки (строгое совпадение).
- $[\dots]$ — разрешены русские и английские буквы, пробелы и дефис.
- $\{2,50\}$ — ограничение длины от 2 до 50 символов.
- Проходят: Иван, Анна-Maria. Не проходят: А (коротко), Иван123 (цифры).

- E-mail адрес:

$\wedge[a-zA-Z0-9.\%+-]+\@[a-zA-Z0-9.-]+\.[a-zA-Z]\{2,\}\$$

- $[a-zA-Z0-9.\%+-]+\$ — имя ящика: буквы, цифры и спецсимволы.
- $\@$ — обязательный символ "собаки".
- $\.[a-zA-Z]\{2,\}$ — доменная зона (точка и минимум 2 буквы).
- Проходят: test@mail.ru. Не проходят: test.ru (нет @).

- Номер телефона: $\wedge\backslash+7\ \backslash(\backslash d\{3\}\backslash)\ \backslash d\{3\}-\backslash d\{2\}-\backslash d\{2\}\$$

- $\backslash+7\ \backslash($ — экранированные символы кода страны и открывающей скобки.
- $\backslash d\{3\}$ — ровно 3 цифры (код оператора).
- $\backslash d\{3\}-\backslash d\{2\}-\backslash d\{2\}$ — блоки цифр, разделенные дефисами.
- Проходят: +7 (999) 123-45-67. Не проходят: 89991234567 (неверный формат).

- Дата начала обучения: $\wedge(0[1-9][12][0-9]|3[01])(0[1-9]|1[0-2])(20[0-2][0-9]|2030)|$

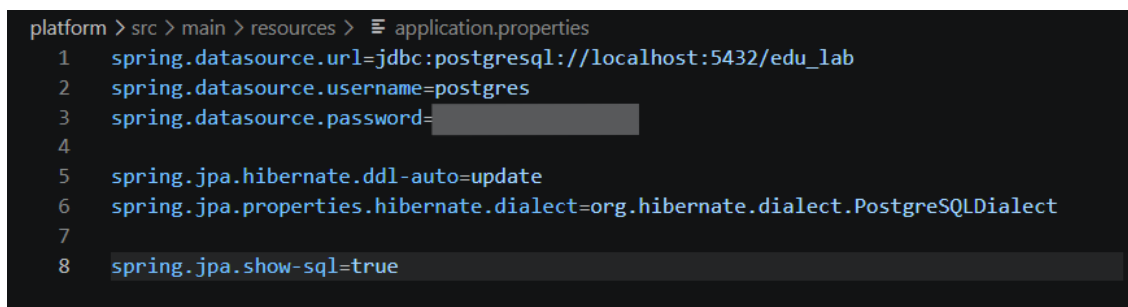
- $(0[1-9][12][0-9]|3[01])|$ — валидация дня (от 01 до 31).
- $(0[1-9]1[0-2])|$ — валидация месяца (от 01 до 12).
- $(20[0-2][0-9]2030)|$ — ограничение года строго от 2000 до 2030.
- Проходят: 27.03.2026. Не проходят: 32.01.2024 (неверный день).

2 Задание 3. Интеграция базы данных, ORM и маршрутизация

В данном разделе реализовано подключение приложения к СУБД PostgreSQL, внедрение ORM-системы на базе Spring Data JPA и настройка динамического рендеринга данных на стороне сервера.

2.1 Настройка подключения и подход Code First

Для взаимодействия с базой данных использовался паттерн проектирования Code First. В конфигурационном файле `application.properties` заданы параметры подключения к PostgreSQL и активировано свойство `spring.jpa.hibernate.ddl-auto=update`. Это позволило фреймворку автоматически генерировать и обновлять структуру реляционных таблиц на основе Java-классов без ручного написания DDL-скриптов.

A screenshot of a code editor showing the content of the application.properties file. The text is as follows:

```
platform > src > main > resources > application.properties
1  spring.datasource.url=jdbc:postgresql://localhost:5432/edu_lab
2  spring.datasource.username=postgres
3  spring.datasource.password=
4
5  spring.jpa.hibernate.ddl-auto=update
6  spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
7
8  spring.jpa.show-sql=true
```

Рис. 4: Скриншот 4: Конфигурация подключения и настройка Hibernate

2.2 Разработка моделей и репозиториев (JPA)

Была создана доменная сущность `Course`, размеченная аннотациями `@Entity` и `@Table`. Для абстрагирования логики доступа к данным (Data Access Object) создан интерфейс `CourseRepository`, наследующий `JpaRepository`. Это обеспечило приложение готовыми методами CRUD (Create, Read, Update, Delete) без необходимости написания SQL-запросов вручную.

```

1  package com.edu.platform.models;
2
3  import jakarta.persistence.*;
4
5  @Entity
6  @Table(name = "courses")
7  public class Course {
8
9      @Id
10     @GeneratedValue(strategy = GenerationType.IDENTITY)
11     private Long id;
12
13     @Column(nullable = false, length = 150)
14     private String title;

```

Рис. 5: Скриншот 5: Реализация сущности Course с аннотациями JPA

2.3 Маршрутизация (Routing) и контроллеры

Для выполнения требований по добавлению новых роутов маршрутизация приложения была расширена. Настроен контроллер `CourseController`, обрабатывающий GET-запросы по пути `/courses`. Контроллер внедряет в себя репозиторий, извлекает список курсов из базы данных и передает его в представление через объект `Model`.

2.4 Динамический рендеринг представлений (Thymeleaf)

Статическая верстка прайс-листа была заменена на динамическую генерацию. С помощью шаблонизатора Thymeleaf и атрибута `th:each` реализован цикл, который проходит по переданной коллекции курсов и автоматически формирует строки CSS Grid таблицы, подставляя в них данные из БД.

```

387 <div class="table-row" role="row" th:each="course : ${coursesList}">
388   <div class="table-cell" role="cell" data-label="Курс:" th:text="${course.title}"></div>
389   <div class="table-cell" role="cell" data-label="Формат:" th:text="${course.format}"></div>
390   <div class="table-cell" role="cell" data-label="Длительность:"
391     th:text="${course.durationMonths + ' мес.'}"></div>
392   <div class="table-cell" role="cell" data-label="Цена:" th:text="${course.price + ' Р'}"></div>
393 </div>

```

Рис. 6: Скриншот 6: Динамическая генерация таблицы прайс-листа

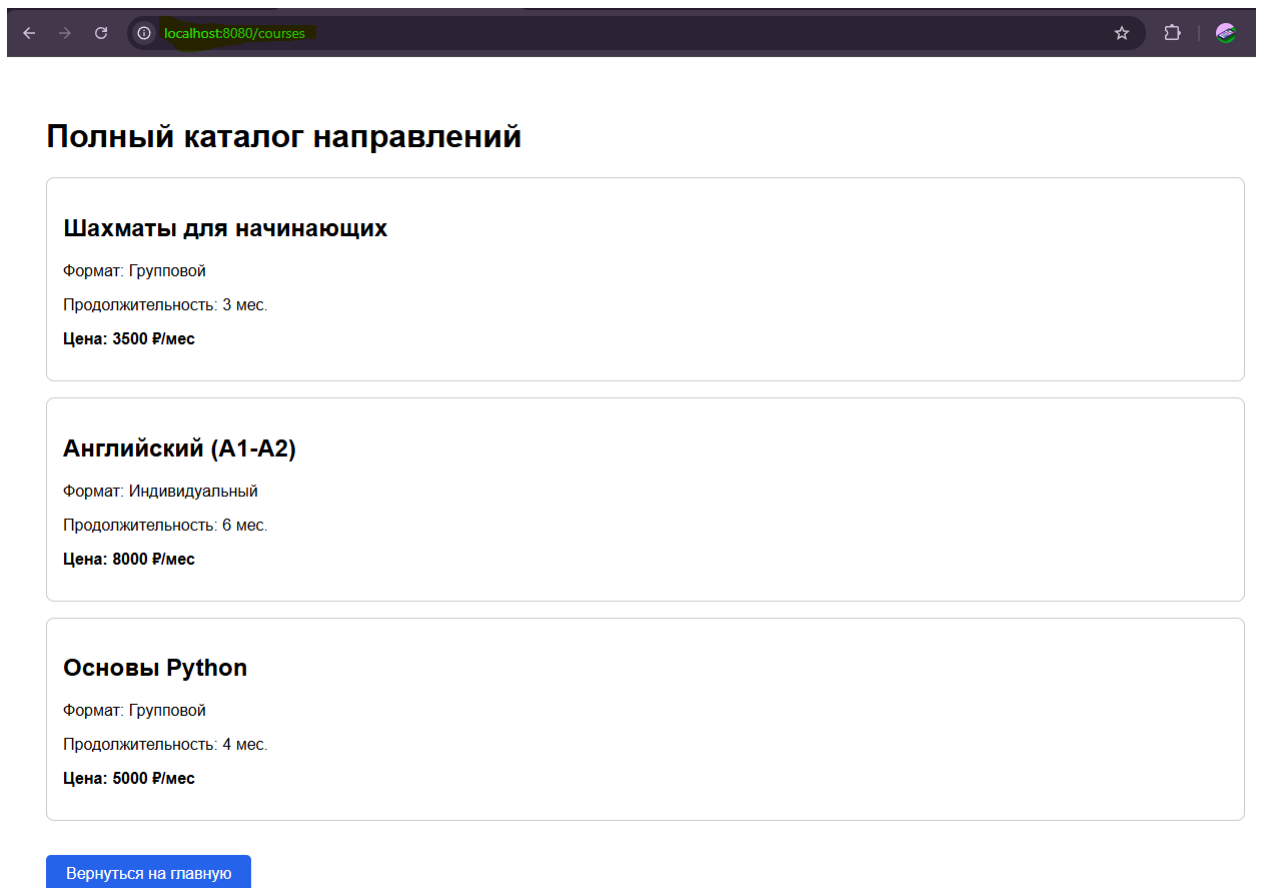


Рис. 7: Скриншот 7: Результат динамического вывода данных из БД на страницу

3 Задания 4 и 5. Безопасность, роли и управление состоянием

На завершающем этапе разработки была реализована подсистема аутентификации и авторизации пользователей, а также механизм сохранения состояния сеанса, что удовлетворяет требованиям к ограничению доступа и работе с сессиями.

3.1 Интеграция Spring Security и аутентификация

Для обеспечения безопасности приложения был подключен модуль `spring-boot-starter-security`. В базе данных создана таблица `users` (модель `User`), хранящая логины, зашифрованные пароли и роли пользователей (например, `ROLE_ADMIN`, `ROLE_USER`).

Шифрование паролей осуществляется с использованием стойкого

криптографического алгоритма `BCryptPasswordEncoder`. Для валидации учетных данных был реализован сервис `CustomUserDetailsService`, переопределяющий метод `loadUserByUsername`, который осуществляет поиск пользователя в базе данных через `UserRepository`.

```
30  @Bean
31  public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
32      http
33          .authorizeHttpRequests(auth -> auth
34              .requestMatchers("/").permitAll()
35              .requestMatchers("/courses").hasAnyRole("USER", "ADMIN")
36          )
37          .formLogin(form -> form
38              .permitAll()
39              .defaultSuccessUrl("/", true)
40          )
41          .logout(logout -> logout
42              .logoutSuccessUrl("/")
43              .permitAll()
44          );
45
46      return http.build();
47  }
48 }
```

Рис. 8: Скриншот 8: Конфигурация цепочки фильтров безопасности (`SecurityFilterChain`)

3.2 Авторизация и ограничение доступа по ролям

В конфигурационном классе `SecurityConfig` были настроены правила авторизации (URL Authorization). Применено строгое разделение прав:

- Маршрут `/` (Landing Page) доступен всем посетителям сайта (`permitAll()`).
- Маршрут `/courses` (Каталог курсов) защищен и доступен только аутентифицированным пользователям с правами `USER` или `ADMIN` (`hasAnyRole()`).

Попытка доступа неавторизованного пользователя к защищенным ресурсам автоматически перенаправляет его на форму логина.

3.3 Работа с состоянием приложения (Session & Cookies)

Для реализации сохранения состояния (Задание 5) использованы механизмы HTTP-сессий и Cookie. При успешной аутентификации Spring Security создает серверную сессию (`HttpSession`) и отправляет браузеру идентификатор `JSESSIONID` в виде Cookie.

Для демонстрации работы состояния на клиентской части был интегрирован диалект `thymeleaf-extras-springsecurity6`.

```
298 <div>
299 <a sec:authorize="isAnonymous()" th:href="@{/login}" class="btn">Войти</a>
300
301 <div sec:authorize="isAuthenticated()" style="display: flex; align-items: center; gap: 1rem;
302 <span style="font-weight: 600; color: var(--primary-color);">
303   Привет, <span sec:authentication="name">Юзеп</span>.</span>
304 </div>
305 <form th:action="@{/logout}" method="post" style="margin: 0;">
306   <button type="submit" class="btn"
307     style="background-color: #dc2626; padding: 0.5rem 1rem;">Выйти</button>
308 </form>
309 </div>
310
311
312 </div>
```

Рис. 9: Скриншот 9: Динамическое изменение интерфейса на основе состояния сессии

Навигационное меню динамически реагирует на статус сессии:

- При отсутствии сессии (`isAnonymous()`) отображается кнопка «Войти».
- При наличии валидной сессии (`isAuthenticated()`) отображается имя текущего пользователя (`sec:authentication="name"`) и кнопка «Выйти».

Выход из системы (`/logout`) реализован через метод `POST` (для защиты от CSRF-атак). При выходе сессия инвалидируется, а Cookie затирается, возвращая приложение в исходное состояние.

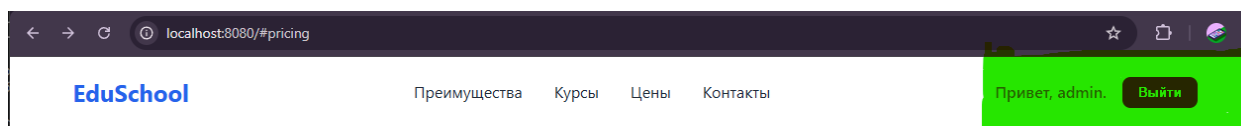


Рис. 10: Скриншот 10: Интерфейс приложения после успешной аутентификации пользователя